Learning USV Drift from Telemetry: Linear, LSTM and Transformer Models

Jake Cushway
University of Victoria
Victoria, Canada
jakecush1@uvic.ca

Rowan Hall
University of Victoria
Victoria, Canada
rowanhall@uvic.ca

Abstract

A shore pilot keeps the Open Ocean Robotics DataXplorer™ USV within 10 nm of a retired NOAA buoy, letting it drift with the motor off and correcting it before it nears the boundary. Knowing where the vessel will be in an hour is the first step toward predicting *when* the pilot must act. We formalise drift prediction as regression from the last 60 minutes of telemetry (position and wind) to the displacement over the next 60 minutes, and compare a persistence baseline, linear regression, an LSTM and a transformer under a deployment-wise split. Code: https://github.com/jakecush1/autonomous-pilot-ML (`usv-intervention/v1_ml/rowan`).[1]

Keywords

Uncrewed surface vehicle, drift prediction, time series regression, LSTM, transformer

1 Problem Definition

Let a deployment be a 1-minute telemetry sequence $\mathcal{S} = \{s_\tau\}_{\tau=1}^{T}$ with $s_\tau = (\phi_\tau, \lambda_\tau, v_\tau, \theta_\tau, m_\tau)$: latitude, longitude, wind speed (kn), the compass direction the wind blows *from*, and motor power (W). A minute is *drifting* if $m_\tau = 0$. Given a time t at which the vessel has drifted for the past H minutes, we want a function

$$f : \mathbb{R}^{H\times 4} \to \mathbb{R}^2, \qquad \hat{\mathbf{y}}_t = f(\mathbf{X}_t), \tag{1}$$

where $\mathbf{X}_t$ is the past window (§2.1) and $\mathbf{y}_t = \mathbf{p}_{t+H} - \mathbf{p}_t \in \mathbb{R}^2$ is the east/north displacement in metres over the next H minutes, assuming the motor stays off. The predicted position is $\hat{\mathbf{p}}_{t+H} = \mathbf{p}_t + \hat{\mathbf{y}}_t$. This is regression, not classification: the output is a continuous vector. We use $H = 60$.

2 Proposed Approach

2.1 Representation

Position. Each fix is projected to metres east/north of the station centre (ϕ_0, λ_0) with an equirectangular projection, with $<$ 0.3% distance error within 10 nm: $\mathbf{p}_\tau = R\,\big((\lambda_\tau - \lambda_0)\cos\phi_0,\ \phi_\tau - \phi_0\big)$, $R = 6371\,\text{km}$, angles in radians. **Wind.** Direction is averaged as $(\sin\theta, \cos\theta)$ so it is continuous across north. The wind pushes the vessel the opposite way, so the push vector is $\mathbf{w}_\tau = -v_\tau(\sin\theta_\tau, \cos\theta_\tau)$ (east, north). **Window.** With $\Delta\mathbf{p}_\tau = \mathbf{p}_\tau - \mathbf{p}_{\tau-1}$, the input is the sequence $\mathbf{X}_t = [\ (\Delta\mathbf{p}_\tau, \mathbf{w}_\tau)\ ]_{\tau=t-H+1}^{t} \in \mathbb{R}^{H\times 4}$.

2.2 Preprocessing and Sample Selection

Gaps of ≤ 5 missing minutes are linearly interpolated (motor power is forward-filled). A minute is *valid* if $m_\tau = 0$, all features exist, and $\|\Delta\mathbf{p}_\tau\| < 93\,\text{m}$ (about 3 kn; faster means towing or transit, not drift). Time t yields a sample $(\mathbf{X}_t, \mathbf{y}_t)$ only if all $2H$ minutes $t - H + 1, \ldots, t + H$ are valid, so neither the history nor the target contains motorised movement. From five deployments (Apr–Oct 2026, 85,910 minutes) this gives 23,319 samples.

Split by deployment. Neighbouring samples share $H - 1$ of their H minutes, so a random split leaks targets into training. We instead hold out whole deployments: train (Apr, May–Jun, Aug; 16,471), validation (Oct 3–13; 2,289), test (Oct 16–23; 4,559). Inputs and targets are standardised with training statistics only.

2.3 Models

All models implement f in Eq. 1. **Persistence** (baseline): $\hat{\mathbf{y}}_t = \sum_\tau \Delta\mathbf{p}_\tau$, i.e., the next hour repeats the last; it captures current and tide without measuring them. **Linear regression**: $\hat{\mathbf{y}}_t = \mathbf{A}\,\mathbf{z}_t + \mathbf{c}$ on the summary $\mathbf{z}_t = \big(\sum_\tau \Delta\mathbf{p}_\tau,\ \mathbf{w}_t,\ \frac{1}{H}\sum_\tau \mathbf{w}_\tau\big) \in \mathbb{R}^6$, fitted by least squares. **LSTM** [2]: two stacked layers (64 hidden units, dropout 0.2) read $\mathbf{X}_t$ minute by minute; $\hat{\mathbf{y}}_t = \mathbf{W}\mathbf{h}_t + \mathbf{b}$ from the last hidden state. Unlike the summary $\mathbf{z}_t$, it sees whether drift is turning or slowing. **Transformer** [6]: each minute is embedded, $\mathbf{e}_\tau = \mathbf{W}_e\mathbf{x}_\tau + \text{PE}(\tau) \in \mathbb{R}^{64}$ with sinusoidal positions, passed through two encoder layers (4 heads, feed-forward 256, dropout 0.1), mean-pooled over time and mapped linearly to $\hat{\mathbf{y}}_t$. Self-attention lets any minute attend to any other directly.

Why these models. Drift is driven mostly by current and tide, which change slowly, so the recent track is the strongest input and persistence is the bar a model must clear. Linear regression tests whether a fixed weighting of past drift and wind adds to it, and its weights are interpretable. The LSTM and transformer test whether the *shape* of the last hour (turning, accelerating, wind shifts) carries information that a single sum loses: the LSTM by recurrence, the transformer by attention. The progression goes from the simplest to the most expressive model, and each must beat the one before it on unseen data to justify its cost. Table 1 lists the settings.

2.4 Training and Inference

Neural models minimise mean squared error on standardised targets with Adam [3] (learning rate 10^{-3}, batch 256, gradient clipping 1.0) for up to 80 epochs. The weights with the lowest validation loss are kept, and training stops after 10 epochs without improvement; with few independent hours of drift, early stopping is the main guard against overfitting. All seeds are fixed to 0. At inference, $\hat{\mathbf{y}}_t$ is de-standardised and added to $\mathbf{p}_t$. We score the endpoint error $e_t = \|\hat{\mathbf{p}}_{t+H} - \mathbf{p}_{t+H}\|_2$ (median and 90th percentile, in metres); the 90th percentile is a candidate safety radius for the pilot.

[1] AI use (Guideline D): the code and a first draft of this paper were generated with Claude (Anthropic) and reviewed, run and edited by the authors.

Table 1: Hyperparameters (all in the code's top-level constants).

Shared	$H = 60$ min; gap fill ≤ 5 min; tow cut-off 93 m/min
LSTM	2 layers, 64 hidden, dropout 0.2
Transformer	$d = 64$, 4 heads, 2 layers, FF 256, dropout 0.1
Training	Adam, lr 10^{-3}, batch 256, clip 1.0, ≤ 80 epochs
Stopping	patience 10 on validation MSE; seed 0

Algorithm 1 Drift prediction: data, training, inference

Input: deployments $\{\mathcal{S}_k\}$, horizon H, model f_Θ
Output: trained f_{Θ^*}; predicted position $\hat{\mathbf{p}}_{t+H}$

1: **for all** deployment $\mathcal{S}_k$ **do**
2: fill gaps ≤ 5 min; $\mathbf{p}_\tau \leftarrow \text{proj}(\phi_\tau, \lambda_\tau)$
3: $\mathbf{w}_\tau \leftarrow -v_\tau(\sin\theta_\tau, \cos\theta_\tau)$
4: $\text{valid}_\tau \leftarrow (m_\tau = 0) \wedge \|\Delta\mathbf{p}_\tau\| < 93$
5: **for all** t with $\text{valid}_{t-H+1..t+H}$ **do**
6: add $(\mathbf{X}_t,\ \mathbf{y}_t = \mathbf{p}_{t+H} - \mathbf{p}_t)$ to split of $\mathcal{S}_k$
7: standardise with train statistics; seed $\leftarrow 0$; best $\leftarrow \infty$
8: **for** epoch = 1..80 **do**
9: **for all** mini-batch $B \subset$ train **do**
10: $\Theta \leftarrow \text{Adam}\big(\Theta, \nabla_\Theta \frac{1}{|B|}\sum_B \|f_\Theta(\mathbf{X}) - \mathbf{y}\|^2\big)$
11: **if** val loss < best **then** best $\leftarrow$ val loss; $\Theta^* \leftarrow \Theta$
12: **else if** no improvement for 10 epochs **then break**
13: **return** $\hat{\mathbf{p}}_{t+H} = \mathbf{p}_t + f_{\Theta^*}(\mathbf{X}_t)$

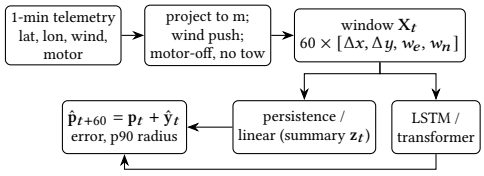


Figure 1: Pipeline: telemetry is projected and filtered, cut into 60-minute motor-off windows, and mapped by each model to the displacement over the next 60 minutes.

3 Implementation

The code is in Python with scikit-learn and PyTorch, organised by pipeline stage: `data_prep.py` (representation, sample selection, split, error metric; Algorithm 1 lines 1–6), `linear_reg.py` (baselines and linear regression), `nn_models.py` (LSTM, transformer and the shared training loop; lines 7–13) and `compare.py` (results table and chart). Every script writes to `outputs/`. The README covers the overview, installation, quick start and results; dependencies are pinned in `requirements.txt` and the data are the five 1-minute exports in the repository, so nothing needs downloading. Quick start:

```
git clone https://github.com/jakecush1/autonomous-pilot-ML
cd autonomous-pilot-ML/usv-intervention/v1_ml/rowan
python3.13 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python data_prep.py && python linear_reg.py
python nn_models.py lstm && python nn_models.py transformer
python compare.py
```

Reproducibility. Seeds for NumPy and PyTorch are fixed to 0, the split is fixed by deployment, and scaling uses training statistics only, so re-running the quick start on CPU gives the numbers in Table 2 exactly; we checked this by running it twice.

Table 2: Sanity run: endpoint error after 60 min (m) on the held-out test deployment. Full evaluation is Milestone 4.

Model	Median	90th pct.	R^2
No drift ($\hat{\mathbf{y}} = 0$)	586	1515	0.00
Persistence	294	570	0.80
Linear regression	318	631	0.77
LSTM	247	497	0.85
Transformer	**246**	**477**	**0.86**

A first run (Table 2) shows the pipeline works end to end. Persistence halves the no-drift error, linear regression does not beat it on the unseen deployment, and both sequence models do, by about 16%. Evaluation over several seeds and horizons, and turning $\hat{\mathbf{p}}_{t+H}$ into a time to boundary in the spirit of Dugan and Utne [1] and an intervention probability [4], follow in Milestone 4. Song et al. [5] correct a physics model with a neural network; we learn drift from telemetry alone.

References

[1] Spencer August Dugan and Ingrid Bouwer Utne. 2024. Development of a Risk Indicator for Ship Drifting Groundings. In *Proc. PSAM17 & ASRAM2024.* https://www.iapsam.org/PSAM17/program/Papers/PSAM17&ASRAM2024-1377.pdf

[2] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Comput.* 9, 8 (1997), 1735–1780. doi:10.1162/neco.1997.9.8.1735

[3] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *Proc. ICLR 2015.* https://arxiv.org/abs/1412.6980

[4] Susanna Dybwad Kristensen, Yiliu Liu, and Ingrid Bouwer Utne. 2022. Dynamic Risk Analysis of Maritime Autonomous Surface Ships. In *Proc. PSAM 16.* https://www.iapsam.org/PSAM16/papers/SU247-PSAM16.pdf

[5] Yuxuan Song et al. 2024. The Prediction and Dynamic Correction of Drifting Trajectory for Unmanned Maritime Equipment Based on Fully Connected Neural Network (FCNN) Embedding Model. *J. Mar. Sci. Eng.* 12, 12 (2024), 2262. doi:10.3390/jmse12122262

[6] Ashish Vaswani et al. 2017. Attention Is All You Need. In *Proc. NeurIPS 2017.* 5998–6008.